CMPS344 Applied Data Science

Spring 2026

1. Project Overview

This project requires students to design, implement, and evaluate a supervised classification solution using real-world data. The workflow must:

- Define a clear business problem and stakeholder.
- Acquire and integrate data from at least three reliable sources.
- Perform thorough data validation and cleaning with documentation.
- Apply feature transformation and engineering with justification.
- Conduct exploratory data analysis (EDA) with meaningful visualizations and dashboards.
- Build at least five classification models, including one baseline.
- Track experiments using MLflow (metrics, hyperparameters, artifacts, versions).
- Evaluate models using technical and business-oriented metrics.
- Explain how predictions support stakeholder decision-making.
- Follow software engineering practices: clean code, structured organization, logging, dependency management via Poetry, and Git version control.
- Implement testing, automation, and CI pipelines via GitHub Actions.

2. Team Requirements

- Teams: 3–4 students.
- Team numbers assigned during formation; include in all submissions.
- Each member must contribute meaningfully to all parts of the project; the final report must document contributions.

3. Dataset Requirements

3.1 Minimum Criteria

- **Rows:** ≥ 3,000
- **Features:** ≥ 7
- **Target variable:** Clearly defined classification target
- **Source:** Real-world data

3.2 Data Source Policy

- The majority of data must be real-world.
- Synthetic datasets not permitted (except oversampling techniques like SMOTE for imbalance, applied only to the training split).

- Dataset must demonstrate business impact.

4. Deliverables and Schedule

Phase	Description	Week Due	Deadline
Phase 1	Team Formation	Week 4	7 Mar 2026, 11:59 PM
Phase 2	Dataset Validation Report & Proposal	Week 7	28 Mar 2026, 11:59 PM
Phase 3	Final Delivery	Week 12	28 Apr 2026, 11:59 PM

Phase 2: Proposal Requirements

- Problem statement: clear business problem, stakeholder, and justification for classification.
- Dataset documentation: name, source URL, citation, feature descriptions, engineered features, class distribution, target definition.
- Full data validation report: row/column counts, missing values, duplicates, data types, quality issues.

Phase 3: Final Delivery Requirements

- Submit a zipped folder with the final PDF report, offline runnable code, and Git repository link.
- Repository must include all code, pyproject.toml, poetry.lock, README.md.

5. Final Report Structure

5.1 Problem Definition & Business Context

- Define business problem, stakeholder, classification justification, decision-making impact.

5.2 Data Acquisition & Integration

- Snapshot of acquisition pipeline.
- Source descriptions with URLs/citations.
- Merge strategy (if multiple data sources are used) and counts before/after merging.

5.3 Data Validation & Documentation

- Snapshot of validation pipeline.
- Validation report: missing values, duplicates, types, class distribution.
- Target variable definition.
- Quality issues and removal justification.

5.4 Preprocessing, Transformation & Engineering

- Snapshot of preprocessing pipeline.
- Cleaning steps with justification.
- Outlier detection and handling.
- Feature transformation (encoding, normalization, etc.) with **rationale**.
- Feature engineering details.
- Feature selection with **rationale**.
- Train/validation/test split procedure.
- Data balancing strategy, if any.
- Before and after summaries.

5.5 Exploratory Data Analysis (EDA)

- At least five meaningful visualizations with interpretation.
- Feature-to-target analysis.
- Insights influencing modeling.
- Dashboard with EDA findings, model comparisons, and business insights.

5.6 Model Development

- At least five models, including baseline.
- Model selection rationale.
- Training and hyperparameter tuning.
- Validation strategy.

5.7 Experiment Tracking with MLflow

- All five (or more) model runs must be logged in MLflow.
- Each run must log: model name, all hyperparameters used, and at least two standard evaluation metrics and two business-related ones.
- Trained models must be saved as MLflow artifacts.
- The report must include a screenshot of the MLflow experiment comparison interface showing all runs side by side.

5.8 Testing

- Develop comprehensive unit tests covering all critical functions and modules to ensure code correctness and robustness.
- Justify the selection of test cases based on potential failure points and critical business logic.
- Implement integration tests to verify the interaction between different components and data flow.
- Measure and report test coverage to identify untested parts of the codebase.
- Provide detailed testing reports including test results, coverage statistics, and any identified issues or bugs.

5.9 Results & Evaluation

- Model performance on training and test data (to detect overfitting).
- At least two standard evaluation metrics and two business-related ones per model, and justify your choice relative to the business problem (e.g., if false negatives are costly, recall and F1 matter more than accuracy)
- A comparison table or chart of all models.
- Error analysis: where does the best model fail? What types of errors does it make?
- Business-oriented interpretation: what do the results mean for the stakeholder? What actions do they enable?
- Discussion of any unsuccessful approaches tried and why they were abandoned.

5.10 Code Automation

- Use Makefiles to automate common tasks such as data preprocessing, model training, evaluation, and testing.
- Employ configuration files like `.env` to securely load environment-specific settings, including API keys, database credentials, and other sensitive parameters.
- Automate the execution of testing suites to ensure code correctness and robustness.
- Integrate automation scripts with continuous integration pipelines to streamline development workflows.
- Document automation processes clearly for reproducibility and team collaboration.

5.11 Continuous Integration

- Implement GitHub Actions workflows to automate the build, test, and deployment processes.
- Configure pipelines to run unit and integration tests on every pull request and push to main branches.
- Automate code quality checks such as linting and static analysis within the CI pipeline.
- Document the CI pipeline configuration and usage.

5.12 Limitations & Future Work

- Honest discussion of limitations.
- Suggestions for improvement.

5.13 Team Contributions

- Table of team members and contributions.

6. Code Requirements

- **Language:** Python
- **Dependency management:** Poetry

- **Version control:** Git repository (clean, organized)
- **Reproducibility:** `poetry install` + run instructions must reproduce results.
- **Structure:** Modular, cookie-cutter folder structure.
- **Code quality:** Clean code practices.
- **README:** Team names, project description, setup/run instructions.
- **Testing:** pytest coverage.
- **Automation:** make files, .env files.
- **CI:** GitHub Actions.

7. Grading

7.1 Grade Breakdown

Component	Marks
Problem Formulation & Dataset	8
Data Cleaning, Feature Selection, Transformation, Engineering	8
EDA	6
Modelling	6
Experiment Tracking	6
Evaluation	6
Interpretation of Results	6
Reproducibility (Git + Poetry)	4
Report Quality	6
Clean Code	8
Testing	6
Code Automation	4
GitHub Actions	6
Total	**80**
Individual Participation	**% Total**

7.2 Bonus Marks

Bonus Item	Marks
Deployment (public endpoint)	10
Merging multiple real-world dataset sources meaningfully.	10
Deliver an interactive dashboard like Streamlit, Power BI, Dash, or any equivalent tool) that presents EDA findings, model performance comparisons, and business insights in a readable, non-technical format suitable for a stakeholder audience.	10

8. Academic Integrity

- All work must be original.
- Use of open-source libraries and public datasets permitted with citation.
- Plagiarism will result in penalties, including possible zero grade.

- **Code discussions are extremely important to assess each member's individual work. Failing to explain or modify the code during discussion, or justify the code choices, will be interpreted as AI plagiarism.**